

Module 1: Introduction to SQL & Database Basics

The Evolution of Data Storage

Before we dive into databases and SQL, let's understand why we need them by examining how we store different types of data in our daily lives and work.

Example 1: Shopping List

When you need to buy groceries, you might write a shopping list on paper or use a notes app on your phone.

Storage Method	Advantages	Disadvantages
Paper	Quick, simple, no technology needed	Easy to lose, can't share easily, no backup
Notes App	Always with you, can share, basic backup	Limited features, hard to organize complex lists

Impact of data loss: Minor inconvenience. You might forget to buy milk, but you can always go back to the store.

Example 2: Employee Salary Sheet

A company maintaining employee salary records needs more than just a simple list. This is where Excel becomes useful.

Why Excel over paper?

- Automatic calculations (total salary, deductions, tax)
- Easy to modify and update multiple records
- Can sort and filter data
- Can create charts and summaries
- Digital backup possible

- Less prone to calculation errors

Employee ID	Name	Basic Salary	Allowances	Deductions	Net Salary
101	Amit Kumar	50,000	10,000	5,000	=C2+D2-E2
102	Priya Sharma	60,000	12,000	6,000	=C3+D3-E3

Impact of data loss: Serious problem. Without backups, recreating salary records could take weeks and cause payment delays affecting employees' livelihoods.

Example 3: Amazon's Product Catalog

Amazon manages millions of products with complex relationships: categories, sellers, inventory, prices, reviews, recommendations, etc.

Why not Excel for Amazon?

- **Scale:** Excel can handle thousands of rows, but struggles with millions. Amazon has over 350 million products.
- **Concurrent access:** Excel doesn't handle multiple users editing simultaneously well. Amazon needs thousands of sellers updating products at the same time.
- **Speed:** Finding a specific product in millions of Excel rows would be extremely slow.
- **Data integrity:** No way to ensure data consistency (e.g., prevent negative prices, ensure product codes are unique).
- **Relationships:** Difficult to manage connections between products, reviews, customers, orders, and inventory.
- **Security:** Can't control who can view/modify specific data (e.g., only allow seller to edit their own products).
- **Backup and recovery:** Excel files can corrupt. Critical data needs robust backup systems.

Impact of data loss: Catastrophic. Loss of Amazon's product catalog would mean:

- Complete business shutdown
- Millions of dollars in revenue loss per minute
- Loss of customer trust
- Legal and regulatory consequences

- Potentially impossible to fully recreate without backups

Enter the Database Management System (DBMS)

As data grows in volume and complexity, we need specialized tools to manage it efficiently. This is where a **Database Management System (DBMS)** comes in.

Understanding DBMS Through an Analogy

Think about how you work with documents:

 Note:

Document Processing:

User → Microsoft Word / Adobe Acrobat (Document Processor) → Document File (.docx / .pdf)

Database Processing:

User → DBMS (SQL Server / MySQL / Oracle) → Database → Database Files (.mdf / .ldf)

Just as you don't manually edit the binary content of a .docx file (it would be impossible to understand), you don't directly modify database files on the hard drive. Instead, you use a DBMS that:

- Provides a user-friendly interface to interact with data
- Manages the complex file storage behind the scenes
- Ensures data integrity and consistency
- Optimizes storage and retrieval operations
- Handles concurrent access from multiple users

What is a DBMS?

A **Database Management System (DBMS)** is software that provides an interface to create, store, retrieve, update, and manage data in a database. It acts as an intermediary between users/applications and the actual data files.

Key Features of a DBMS

Feature	Description	Real-World Benefit
SQL Support	Standardized query language	Write complex queries to extract exactly the data you need in seconds
Concurrency Control	Multiple users can access data simultaneously	Thousands of Amazon sellers can update products at the same time without conflicts
Transactions	Groups of operations that succeed or fail together	When you transfer money, both debit and credit must happen, or neither happens
Access Control	Fine-grained permissions system	Cashiers can view prices but only managers can change them
Data Integrity	Constraints ensure data validity	Prevent impossible data like negative age or duplicate employee IDs
Performance	Optimized indexing and query execution	Search through millions of records in milliseconds
Backup & Recovery	Automated backup and point-in-time recovery	Restore data to any point in time if something goes wrong
Scalability	Handle growing data volumes	System grows from thousands to billions of records without redesign

DBMS vs RDBMS: Understanding the Difference

What is a DBMS?

A DBMS (Database Management System) is a general term for any software that manages databases. It can store data in various formats without necessarily maintaining relationships between different data entities.

What is an RDBMS?

An RDBMS (Relational Database Management System) is a type of DBMS that specifically stores data in **tables** (relations) and manages **relationships** between these tables. This is the most common type of database system used in business applications.

Aspect	DBMS	RDBMS
Data Storage	Files, hierarchical, network	Tables with rows and columns
Relationships	Limited or no relationship support	Foreign keys establish relationships between tables
Normalization	Not supported	Supports normalization to reduce redundancy
ACID Properties	May not fully support	Fully supports ACID (Atomicity, Consistency, Isolation, Durability)
Examples	XML databases, file systems	SQL Server, MySQL, Oracle, PostgreSQL
Best For	Simple data storage, small applications	Complex applications with related data

Introduction to Non-Relational Databases (NoSQL)

While RDBMS systems work well for structured data with clear relationships, some modern applications require different approaches. **Non-relational databases (NoSQL)** are designed for specific use cases:

Database Type	Use Case	Examples
Document Stores	Semi-structured data, flexible schemas	MongoDB, CouchDB
Key-Value Stores	High-speed caching, session management	Redis, Amazon DynamoDB
Column-Family	Big data analytics, time-series data	Apache Cassandra, HBase
Graph Databases	Social networks, recommendation engines	Neo4j, Amazon Neptune

 Note:

This course focuses on RDBMS and SQL because relational databases remain the foundation of most business applications and are essential knowledge for any data professional. Once you master SQL and relational concepts, learning NoSQL databases becomes much easier.

What is SQL?

SQL (Structured Query Language) is the standardized programming language used to communicate with relational database management systems. It allows you to:

- Create and modify database structures (tables, views, indexes)
- Insert, update, and delete data
- Query data with powerful filtering and aggregation
- Control access and security
- Manage transactions and ensure data consistency

Why Learn SQL?

- **Universal language:** Works across different database platforms (SQL Server, MySQL, Oracle, PostgreSQL)

- **Industry standard:** Required skill for data analysts, database administrators, backend developers, and data scientists
- **Powerful and efficient:** Perform complex operations that would take hours in Excel in seconds
- **Declarative:** You specify *what* you want, not *how* to get it (the DBMS optimizes execution)
- **Career advancement:** One of the most in-demand technical skills across industries

SQL vs Excel: A Real-World Comparison

Let's examine a practical scenario that demonstrates why SQL is essential for complex data operations.

Scenario: Employee Performance Analysis

You need to generate a report that shows:

- All employees who joined in the last 2 years
- Working in the Sales or Marketing departments
- Who have completed at least 5 projects
- With an average project rating above 4.0
- Show their total sales amount
- Grouped by department and region
- Sorted by performance score (calculated from multiple factors)

Excel Approach

1. Open employees.xlsx, projects.xlsx, sales.xlsx, departments.xlsx
2. Use VLOOKUP to match employees with departments (prone to errors)
3. Use VLOOKUP to match employees with their projects
4. Manually filter for join date in last 2 years
5. Filter for Sales/Marketing departments
6. Use COUNTIF to count projects per employee
7. Use AVERAGEIF to calculate average ratings
8. Create helper columns for calculations
9. Use complex IF formulas for conditional logic
10. Use SUMIF for sales totals
11. Create pivot tables for grouping
12. Manual sorting and formatting

Time taken: 30-60 minutes (for experienced user)
 Error prone: Yes (formulas break if data structure changes)
 Maintainability: Poor (needs manual update each time)

SQL Approach

```
SELECT
    d.DepartmentName,
    d.Region,
    e.EmployeeName,
    e.JoinDate,
    COUNT(p.ProjectID) AS TotalProjects,
    AVG(p.Rating) AS AvgRating,
    SUM(s.SalesAmount) AS TotalSales,
    (AVG(p.Rating) * COUNT(p.ProjectID) * SUM(s.SalesAmount) / 10000) AS PerformanceScore
FROM Employees e
INNER JOIN Departments d ON e.DepartmentID = d.DepartmentID
INNER JOIN Projects p ON e.EmployeeID = p.EmployeeID
INNER JOIN Sales s ON e.EmployeeID = s.EmployeeID
WHERE e.JoinDate >= DATEADD(YEAR, -2, GETDATE())
    AND d.DepartmentName IN ('Sales', 'Marketing')
GROUP BY d.DepartmentName, d.Region, e.EmployeeName, e.JoinDate
HAVING COUNT(p.ProjectID) >= 5
    AND AVG(p.Rating) > 4.0
ORDER BY PerformanceScore DESC;
```

Time taken: 2-5 minutes (including testing)
 Error prone: No (reusable, consistent results)
 Maintainability: Excellent (runs automatically on updated data)

Output:

DepartmentName	Region	EmployeeName	JoinDate	TotalProjects	AvgRating	TotalSales	PerformanceScore
Sales	North	Rajesh Kumar	2023-03-15	8	4.5	₹12,50,000	4500.00
Marketing	South	Priya Menon	2023-07-20	6	4.7	₹8,75,000	2467.50

DepartmentName	Region	EmployeeName	JoinDate	TotalProjects	AvgRating	TotalSales	PerformanceScore
Sales	West	Amit Shah	2024-01-10	5	4.2	₹6,20,000	1302.00

Key Advantages of SQL

- **Automation:** Save the query as a stored procedure, run it daily automatically
- **Consistency:** Same query always produces consistent results
- **Speed:** Processes millions of records in seconds
- **Precision:** Complex filtering and calculations in a single statement
- **Maintainability:** When data structure changes, update query once
- **Scalability:** Works equally well with 100 or 100 million records

Understanding Relational Databases

Relational databases organize data into tables (also called relations). Each table consists of rows and columns:

- **Table:** A collection of related data organized in rows and columns (e.g., Employees, Products, Orders)
- **Row (Record/Tuple):** Represents a single entry in the table (e.g., one employee, one product)
- **Column (Field/Attribute):** Represents a specific attribute of the data (e.g., EmployeeName, Price)
- **Relationship:** Connections between tables using keys (e.g., Employee works in Department)

Example: Simple Database Structure

Departments Table

DepartmentID	DepartmentName	Location
1	Sales	Mumbai

DepartmentID	DepartmentName	Location
2	IT	Bangalore
3	HR	Delhi

Employees Table

EmployeeID	EmployeeName	DepartmentID	Salary
101	Amit Kumar	1	₹50,000
102	Priya Sharma	2	₹60,000
103	Rahul Verma	1	₹55,000

Notice that the Employees table doesn't store the department name directly. Instead, it stores a `DepartmentID` that references the Departments table. This is called a **relationship** and is a core concept in relational databases.

SQL Server vs MySQL

While SQL is standardized, different database management systems have slight variations in syntax and features. This course focuses on **SQL Server** syntax, which is Microsoft's enterprise-level database system.

Feature	MySQL	SQL Server
Auto-increment	<code>AUTO_INCREMENT</code>	<code>IDENTITY(1,1)</code>
Limit rows	<code>LIMIT n</code>	<code>TOP n</code> or <code>OFFSET/FETCH</code>
Current date/time	<code>NOW()</code>	<code>GETDATE()</code>
String length	<code>LENGTH()</code>	<code>LEN()</code>
Null handling	<code>IFNULL()</code>	<code>ISNULL()</code>

Creating a Database

Before we can store data, we need to create a database. A database acts as a container for multiple tables.

Syntax

```
CREATE DATABASE database_name;
```

Example

```
-- Create a database for a school management system
CREATE DATABASE School;

-- To use this database for subsequent operations
USE School;
```

Output:

Command completed successfully.

Database 'School' created.

Note:

The `USE` statement tells SQL Server which database to work with. All subsequent table operations will be performed in the selected database.

Dropping a Database

To remove a database permanently, use the `DROP DATABASE` command:

```
DROP DATABASE School;
```

⚠ Important:

The `DROP DATABASE` command permanently deletes the database and all its tables. This action cannot be undone. Always ensure you have backups before dropping a database.

Understanding Data Types

Every column in a table must have a data type that defines what kind of data it can store. Choosing the correct data type is important for data integrity and storage efficiency.

Common SQL Server Data Types

Numeric Data Types

Data Type	Description	Example
<code>INT</code>	Whole numbers (integers)	23, -100, 0
<code>DECIMAL(p, s)</code>	Fixed precision numbers. p = total digits, s = decimal places	<code>DECIMAL(5,2)</code> can store 999.99
<code>FLOAT</code>	Approximate numeric values with floating decimal points	3.14159, 2.718
<code>BIT</code>	Boolean values (0 or 1, representing false or true)	0, 1

Character/String Data Types

Data Type	Description	Example
<code>VARCHAR(n)</code>	Variable-length character string (max n characters)	<code>VARCHAR(50)</code> for names

Data Type	Description	Example
CHAR(n)	Fixed-length character string (exactly n characters)	CHAR(10) for codes
TEXT	Large text data (up to 2GB)	Long descriptions, articles

Date and Time Data Types

Data Type	Description	Format
DATE	Stores date only	YYYY-MM-DD (e.g., 2010-06-12)
TIME	Stores time only	HH:MM:SS (e.g., 14:30:00)
DATETIME	Stores both date and time	YYYY-MM-DD HH:MM:SS

Creating Tables

Tables are the fundamental structure in a database where actual data is stored. Each table has a defined schema (structure) that specifies column names and their data types.

Syntax

```
CREATE TABLE table_name
(
    column1 datatype,
    column2 datatype,
    column3 datatype,
    ...
);
```

Example: Creating a Students Table

```
CREATE TABLE Students
(
    Roll INT,
    StudentName VARCHAR(50),
    Marks DECIMAL(5, 2),
    DateOfBirth DATE,
    LastLoginTime TIME
);
```

Output:

Command completed successfully.

Table 'Students' created with 5 columns.

Understanding the Table Structure

Let's break down each column in the Students table:

- `Roll INT` - Stores student roll numbers as whole numbers
- `StudentName VARCHAR(50)` - Stores student names up to 50 characters
- `Marks DECIMAL(5,2)` - Stores marks with up to 5 total digits and 2 decimal places (e.g., 999.99)
- `DateOfBirth DATE` - Stores the student's birth date
- `LastLoginTime TIME` - Stores the last time the student logged in

Viewing Table Structure

To view the structure of a table in SQL Server, use the `sp_help` stored procedure:

```
EXEC sp_help 'Students';
```

Output:

Column Name	Data Type	Length	Nullable
Roll	int	4	yes
StudentName	varchar	50	yes
Marks	decimal	5	yes
DateOfBirth	date	3	yes
LastLoginTime	time	5	yes

Inserting Data into Tables

Once a table is created, we can add data to it using the `INSERT INTO` statement.

Basic INSERT Syntax

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

Example 1: Inserting Complete Data

```
INSERT INTO Students (Roll, StudentName, Marks, DateOfBirth, LastLoginTime)
VALUES (23, 'Arun', 65.50, '2010-06-12', '07:05:00');
```

Output:

(1 row affected)

Example 2: Inserting with NULL Values

`NULL` represents the absence of a value. It is different from zero or an empty string. Use `NULL` when data is unknown or not applicable.

```
INSERT INTO Students (Roll, StudentName, Marks, DateOfBirth, LastLoginTime)
VALUES (45, 'Nidhi', NULL, NULL, '14:12:56');
```

Output:

(1 row affected)

Note:

In this example, Nidhi's marks and date of birth are unknown, so we insert `NULL` values for those columns.

Example 3: Inserting Partial Data

You don't always need to specify all columns. Columns not mentioned will automatically receive `NULL` values (if allowed by the table structure).

```
INSERT INTO Students (Roll, StudentName, Marks)
VALUES (10, 'Krunal', 87.00);
```

Output:

(1 row affected)

Note:

Since DateOfBirth and LastLoginTime were not specified, they will be set to NULL automatically.

Example 4: Inserting Multiple Rows

SQL Server allows you to insert multiple rows in a single statement, which is more efficient:

```
INSERT INTO Students (Roll, StudentName, Marks, DateOfBirth, LastLoginTime)
VALUES
    (15, 'Priya', 92.50, '2010-03-15', '08:30:00'),
    (18, 'Rahul', 78.00, '2010-07-22', '09:15:00'),
    (22, 'Sneha', 85.75, '2010-11-08', '07:45:00');
```

Output:

(3 rows affected)

Important Date and Time Format Rules

- **Date format:** Must be in `YYYY-MM-DD` format (ISO standard)
- **Time format:** Must be in `HH:MM:SS` format (24-hour clock)
- **DateTime format:** `YYYY-MM-DD HH:MM:SS`

Viewing Data from Tables

To view the data stored in a table, use the `SELECT` statement. The asterisk (*) is a wildcard that represents "all columns."

Syntax

```
SELECT * FROM table_name;
```

Example

```
SELECT * FROM Students;
```

Output:

Roll	StudentName	Marks	DateOfBirth	LastLoginTime
23	Arun	65.50	2010-06-12	07:05:00
45	Nidhi	NULL	NULL	14:12:56
10	Krunal	87.00	NULL	NULL
15	Priya	92.50	2010-03-15	08:30:00
18	Rahul	78.00	2010-07-22	09:15:00
22	Sneha	85.75	2010-11-08	07:45:00

Understanding NULL

NULL is a special marker used in SQL to indicate that a data value does not exist in the database. It is important to understand what NULL represents:

- **NULL is not zero** - Zero is a numeric value, NULL means "no value"
- **NULL is not an empty string** - An empty string ("") is still a value, NULL means "unknown"
- **NULL is not the same as a space** - A space is a character, NULL means "absent"

When to Use NULL

- When data is unknown (e.g., a student's marks when they were absent)
- When data is not applicable (e.g., a person's spouse name if they are unmarried)
- When data has not been provided yet

Example Scenarios

Scenario	Value to Use	Reason
Student was absent for exam	NULL	Marks are unknown
Student scored zero	0	This is an actual score
Student name not provided	NULL	Name is unknown
Student pressed spacebar as name	' ' (space)	This is actual input (incorrect, but a value)

Dropping Tables

To permanently remove a table from the database, use the `DROP TABLE` command:

```
DROP TABLE Students;
```

Output:

Command completed successfully.

Table 'Students' has been dropped.

⚠ Important:

The `DROP TABLE` command permanently deletes the table and all its data. This action cannot be undone. Always create backups before dropping important tables.

Finding Tables in the Database

SQL Server provides system views to help you find tables in your database. The most commonly used view is `INFORMATION_SCHEMA.TABLES`.

Example: Search for a Specific Table

```
SELECT *  
FROM INFORMATION_SCHEMA.TABLES  
WHERE TABLE_NAME = 'Students';
```

Output:

TABLE_CATALOG	TABLE_SCHEMA	TABLE_NAME	TABLE_TYPE
School	dbo	Students	BASE TABLE

Example: List All Tables in Current Database

```
SELECT TABLE_NAME  
FROM INFORMATION_SCHEMA.TABLES  
WHERE TABLE_TYPE = 'BASE TABLE';
```

Output:

TABLE_NAME
Students
Teachers
Courses

Best Practices

1. **Use meaningful names:** Choose descriptive names for databases, tables, and columns (e.g., StudentName instead of SN)

2. **Follow naming conventions:** Use PascalCase for table and column names (e.g., StudentName, DateOfBirth)
3. **Choose appropriate data types:** Use INT for whole numbers, VARCHAR for variable text, DATE for dates
4. **Be explicit with column names:** Always specify column names in INSERT statements to avoid errors
5. **Use NULL appropriately:** Only use NULL when data is truly unknown or not applicable
6. **Comment your code:** Use -- for single-line comments and /* */ for multi-line comments
7. **Format SQL consistently:** Use proper indentation and line breaks for readability

Common Mistakes to Avoid

- **Inserting data without specifying columns:** Always list the columns explicitly to avoid data mismatch
- **Wrong date format:** Always use YYYY-MM-DD format for dates
- **Confusing NULL with zero or empty string:** Remember that NULL means "no value"
- **Using reserved keywords as names:** Avoid using SQL keywords (like SELECT, INSERT, etc.) as table or column names
- **Not backing up before DROP operations:** Always create backups before dropping databases or tables

Summary

In this module, we covered the foundational concepts of SQL and database management:

- Understanding SQL and relational databases
- Differences between MySQL and SQL Server syntax
- Creating and managing databases
- Understanding data types and choosing appropriate ones
- Creating table structures
- Inserting data into tables
- Understanding and using NULL values correctly
- Viewing table data and structure
- Finding tables in a database



These fundamentals form the building blocks for all SQL operations. In the next module, we will explore how to query and retrieve data from tables using various filtering and sorting techniques.

End of Module 1: Introduction to SQL & Database Basics

